

Эскалация привилегий

Типовые ошибки конфигурации в Linux и Windows

Курс «Защита операционных систем»

План лекции

1. Что такое эскалация привилегий и почему это вопрос конфигурации
2. Общая методология анализа (recon → misconfig → impact)
3. **Linux**: шесть типовых уязвимостей
4. **Windows**: шесть аналогичных уязвимостей
5. Сопоставление Linux ↔ Windows
6. Общие принципы защиты и инструменты аудита
7. Переход к практической части (лабораторная работа)

Что такое эскалация привилегий

- **Вертикальная эскалация** — пользователь с ограниченными правами получает права более привилегированного субъекта (чаще всего — root / SYSTEM).
- В подавляющем большинстве реальных инцидентов это **не уязвимость в ядре**, а **ошибка конфигурации**:
 - слишком широкие права на файл или объект;
 - привилегированный процесс, который доверяет вводу/путям, контролируемым непривилегированным пользователем;
 - избыточные полномочия, выданные «для удобства».
- Вывод: эскалация привилегий — это в первую очередь предмет **администрирования и аудита**, а не только эксплойтов.

Общая методология

Что бы ни атаковали — Linux или Windows, схема одна:

1. **Разведка (recon)** — какие права есть у текущего пользователя; что запускается с повышенными правами (сервисы, задания по расписанию, SUID-бинари); где привилегированный процесс взаимодействует с объектами, которые мы можем изменить.
2. **Поиск несоответствия (misconfig)** — права на файл/реестр/конфигурацию шире, чем нужно для его роли.
3. **Воздействие (impact)** — заставить привилегированный процесс выполнить код, прочитать/записать файл или предоставить доступ от нашего имени.
4. **Защита** — зеркально: найти то же самое несоответствие **до** того, как это сделает атакующий, и устранить его по принципу наименьших привилегий.

Часть 1

Linux: шесть типовых уязвимостей

Linux: обзор

№	Уязвимость	Суть
1	Опасный SUID	Привилегированный интерпретатор/утилита с SUID-битом
2	SUID + <code>\$PATH</code>	SUID-бинарь вызывает команду без полного пути
3	Опасный sudoers	<code>NOPASSWD</code> на программу с «побегом» в shell
4	root-cron + writable dir	root выполняет скрипт из каталога, доступного для записи
5	Symlink + privileged reader	Привилегированный процесс читает/пишет по пути без проверки symlink
6	systemd unit misconfig	Файл, запускаемый от root через systemd, доступен для записи

1. Опасный SUID-бит

- Бит `setuid` на исполняемом файле → процесс выполняется с правами **владельца файла**, а не того, кто его запустил.
- Опасно, если SUID стоит на программе общего назначения, умеющей выполнять произвольный код: интерпретаторы (`python` , `perl`), редакторы, архиваторы.

Детект:

```
find / -xdev -perm -4000 -type f 2>/dev/null
```

Защита:

- SUID — только на то, что реально требует смены uid (`passwd` , `sudo` , `ping`), и только из официальных пакетов.
- Регулярно сверять список SUID-файлов с эталоном; любое новое совпадение — инцидент.

2. SUID-бинарь + уязвимость `$PATH`

- Собственный SUID-бинарь вызывает внешнюю команду **без полного пути** (`system("id")`, `popen("ls")`).
- Такой вызов ищет команду по переменной `PATH`. Если в начало `PATH` подставить каталог со своим файлом с тем же именем — он выполнится с правами владельца SUID-файла.

Защита:

- В привилегированном коде — только абсолютные пути: `execve("/usr/bin/id", ...)` вместо `system("id")`.
- Явно задавать безопасный `PATH` в начале программы/скрипта, не наследовать его от вызывающего окружения.

3. Опасная запись в sudoers

- `/etc/sudoers` и `/etc/sudoers.d/*` определяют, что пользователь может выполнить через `sudo`, в т.ч. без пароля (`NOPASSWD`).
- Если разрешена программа с интерактивным режимом (`less`, `vi`, `find`, `awk` ...) — пользователь может выйти из неё в shell, унаследовав привилегии `sudo`. Класс техник известен как «GTFOBins для sudo».

Детект: `sudo -l`

Защита:

- Не выдавать `NOPASSWD` на программы с известными «escape»-техниками.
- Сужать разрешённые команды до конкретных аргументов без wildcard; по возможности — `sudoedit`, тег `NOEXEC`.

4. root-cron + каталог, доступный для записи

- `cron` часто выполняет скрипты от root по расписанию.
- Если скрипт лежит в каталоге, куда может писать непривилегированный пользователь, — он подменяет содержимое скрипта, и при следующем срабатывании cron код атакующего выполняется от root.

Детект:

```
cat /etc/crontab /etc/cron.d/*  
find / -xdev -writable -type d 2>/dev/null
```

Защита:

- Скрипты и каталоги, запускаемые от root (cron/systemd/init), — доступ на запись только у root.
- Контроль целостности (контрольные суммы) root-скриптов.

5. Symlink-атака на привилегированный «читатель»

- Привилегированный процесс читает/пишет файл по заранее известному пути, **не проверяя**, что это обычный файл, а не символическая ссылка.
- Непривилегированный пользователь, контролирующий путь, подменяет его симлинком на файл, который ему самому читать/писать не разрешено. Процесс root выполняет операцию «за него» — утечка или порча данных.

Защита:

- Проверка `-L` (не симлинк) непосредственно перед операцией.
- Открытие файлов с `O_NOFOLLOW`; изоляция привилегированных операций от путей, частично контролируемых пользователем.

6. systemd unit с неправильными правами

- systemd-сервисы и таймеры часто выполняются от root.
- Если файл, указанный в `ExecStart=` (или `EnvironmentFile=`, `WorkingDirectory=`), доступен для записи непривилегированному пользователю — он получает выполнение кода от root при следующем запуске юнита.

Детект:

```
systemctl cat <unit>  
systemd-analyze security <unit>
```

Защита:

- Все файлы, упомянутые в юните, — владелец root, права без записи для group/other.
- Сами unit-файлы в `/etc/systemd/system/` — не выше `644`, владелец root.

Часть 2

Windows: шесть аналогичных уязвимостей

Windows: обзор

№	Уязвимость	Аналог из Linux-части
1	Слабые права на бинарь службы	Опасный SUID
2	Unquoted Service Path	SUID + \$PATH
3	AlwaysInstallElevated	Опасный sudoers
4	Задание планировщика от SYSTEM в writable-пути	root-cron + writable dir
5	Junction/symlink-атака (NTFS reparse point)	Symlink + privileged reader
6	Слабые права на раздел реестра службы	systemd unit misconfig

Общий принцип тот же: служба/процесс работает с повышенными правами, а объект, с которым он взаимодействует (файл, путь, реестр), доступен для изменения обычному пользователю.

1. Слабые права на исполняемый файл службы

- Службы Windows (Services) часто выполняются от имени `LocalSystem`.
- Если ACL на exe-файл службы (или на сам объект службы в Service Control Manager) разрешает запись обычному пользователю — он подменяет бинарь; при следующем запуске служба выполняет код атакующего от SYSTEM.

Детект:

```
icacls "C:\Program Files\App\service.exe"  
sc qc <ServiceName>
```

(утилита `accesschk` из Sysinternals — для массового аудита ACL служб)

Защита: запись на бинарь и каталог службы — только Administrators/SYSTEM; не устанавливать сервисы в пользовательские/временные каталоги.

2. Unquoted Service Path

- Если путь к бинарию службы содержит пробелы и не взят в кавычки, например:

```
C:\Program Files\My App\service.exe
```

- Windows пытается интерпретировать путь по частям от каждого пробела:

C:\Program.exe, затем C:\Program Files\My.exe, и т.д. Файл, подложенный в более ранней точке разрыва (если туда разрешена запись), выполнится вместо настоящего — с правами службы.

Детект:

```
wmic service get name,displayname,pathname,startmode |  
findstr /i /v "" | findstr /i /v "C:\Windows\"
```

Защита: все пути с пробелами в ImagePath — строго в кавычках; проверка при каждой установке/обновлении ПО.

3. AlwaysInstallElevated

- Две политики реестра:

```
HKLM\...\Policies\Microsoft\Windows\Installer\AlwaysInstallElevated
```

```
HKCU\...\Policies\Microsoft\Windows\Installer\AlwaysInstallElevated
```

- Если **обе** равны `1`, любой пользователь может установить `.msi`-пакет с правами SYSTEM — вне зависимости от собственных прав. Классический пример «выдали лишние полномочия ради удобства», прямой аналог избыточного `NOPASSWD` в `sudoers`.

Детект:

```
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated  
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
```

Защита: значение должно быть `0` или отсутствовать; контроль через GPO во всём домене.

4. Задание планировщика от SYSTEM в writable-пути

- Task Scheduler (`schtasks`) — прямой аналог cron в Windows.
- Если задание выполняется от SYSTEM, а его «Action» указывает на скрипт/exe в каталоге, доступном для записи обычному пользователю, — подмена файла даёт выполнение кода от SYSTEM при следующем срабатывании.

Детект:

```
schtasks /query /fo LIST /v  
icacls "<путь из Action>"
```

Защита: цели заданий — только в каталогах, доступных на запись исключительно администраторам; регулярный аудит `Get-ScheduledTask` + `Get-Acl` .

5. Junction/symlink-атака (NTFS reparse points)

- NTFS поддерживает reparse points — символические ссылки и junction-точки (`mklink` , `mklink /j`).
- Если привилегированный процесс (служба/SYSTEM) работает с фиксированным путём в каталоге, куда может писать обычный пользователь (временные файлы, логи, «чинит» файлы при обновлении), — путь подменяется junction-точкой на защищённый файл/каталог. Процесс действует уже над реальной целью.
- Этот класс атак использовался в ряде реальных уязвимостей механизмов обновления ПО.

Защита: проверка reparse-point перед операцией (`FILE_FLAG_OPEN_REPARSE_POINT` , валидация через `GetFinalPathNameByHandle`); не выполнять операции от SYSTEM непосредственно в каталогах с «чужим» владельцем записи.

6. Слабые права на раздел реестра службы

- Конфигурация каждой службы хранится в `HKLM\SYSTEM\CurrentControlSet\Services\<имя>` (`ImagePath`, `Start` и т.д.).
- Если ACL этого раздела реестра изменён и разрешает запись не-администратору — атакующий меняет `ImagePath` напрямую, указывая на свой бинарь. Код выполняется от SYSTEM при следующем старте службы/перезагрузке.
- Прямой аналог записываемого systemd unit-файла, только на уровне реестра.

Детект: `accesschk.exe -kvuqsw hk\lm\system\currentcontrolset\services`

Защита: не менять ACL по умолчанию на дереве `...\Services`; периодический аудит через Sysinternals AccessChk.

Сопоставление Linux ↔ Windows

Linux	Windows	Общий корень проблемы
SUID на интерпретаторе	Слабые права на бинарь службы	Привилегированный исполняемый файл доступен для подмены
SUID + <code>\$PATH</code>	Unquoted Service Path	Разрешение имени/пути зависит от окружения, а не от абсолютного адреса
<code>NOPASSWD</code> в sudoers	AlwaysInstallElevated	Администратор выдал избыточные полномочия «для удобства»
root-cron + writable dir	Задание планировщика (SYSTEM) в writable-пути	Планировщик выполняет артефакт из каталога, не защищённого от записи
Symlink + privileged reader	Junction/reparse-point атака	Привилегированный процесс доверяет пути без проверки типа объекта
Писуемый systemd unit	Писуемый раздел реестра службы	Конфигурация привилегированного сервиса доступна для правки

Общие принципы защиты

1. **Наименьшие привилегии** — ничто не должно работать с правами выше необходимого (SUID, SYSTEM-службы, sudo, AlwaysInstallElevated).
2. **Разделение владельца и писателя** — объект (файл/путь/реестр/конфигурация), с которым работает привилегированный процесс, не должен быть доступен для записи менее привилегированному субъекту.
3. **Не доверять окружению** — привилегированный код не должен полагаться на `$PATH`, относительные пути или непроверенные символические ссылки.
4. **Регулярный, а не разовый аудит** — конфигурация «дрейфует» со временем (новые службы, новые cron/Task Scheduler записи); аудит должен быть периодическим и по возможности автоматизированным.
5. **Минимальный образ системы** — меньше установленного ПО и служб — меньше поверхность атаки.

Инструменты аудита (для защитников)

Linux:

- `find ... -perm -4000`, `sudo -l`, `systemd-analyze security` — встроенные
- **Lynis**, **OpenSCAP** — автоматизированный аудит конфигурации
- **LinPEAS** — эnumерация «глазами атакующего» (полезно понимать, что увидит нарушитель)

Windows:

- **Sysinternals Suite**: AccessChk, Autoruns, Process Explorer
- **PowerUp** (PowerSploit) — автоматизированная проверка типовых misconfig'ов служб/задач/реестра
- **WinPEAS**, **Seatbelt** — эnumерация «глазами атакующего»

Справочники конкретных техник: **GTFOBins** (Linux) и **LOLBAS** (Windows) — каталоги

легитимных системных бинарей, которые можно злоупотребить при неверной

Переход к практике

На лабораторной работе вы:

1. Разворачиваете чистую VM **Debian 13** учебным скриптом `lab_setup.sh`, который намеренно создаёт все шесть рассмотренных сегодня linux-уязвимостей (с уникальной меткой `LAV_TOKEN` для каждого студента).
2. По каждому заданию проходите цикл **эксплуатация** → **защита** → **проверка**, подтверждая оба состояния скриншотами с контрольными флагами.
3. Оформляете отчёт по прилагаемому шаблону (`lab_os_protection.md`).

Материал по Windows сегодня — теоретический; аналогичная практическая часть может быть дана отдельно, на изолированном Windows-стенде.

Вопросы?